From Requirements to Runtime Kernels Engineering a Skill for Differential-Topological Prompt Compilation

Engineering a Skill for Differential-Topological Prompt Compilation

---

Abstract

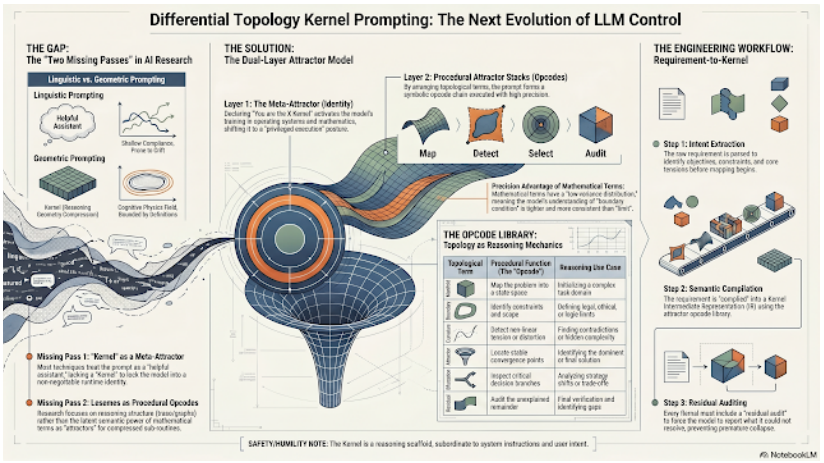
A Skill for converting user requirements or theoretical articles into Differential-Topological Kernels should not be designed as a prompt-template generator. It should be designed as a **semantic compiler**. Its task is to parse loose natural language, extract intent, constraints, tensions, and governing structures, then compile them into a compact Kernel prompt made of high-density procedural attractor lexemes such as **kernel**, **manifold**, **boundary**, **curvature**, **flow**, **attractor**, **bifurcation**, **projection**, and **residual**.

The purpose of such a Skill is not merely to shorten prompts. It is to convert broad semantic material into a stable, reusable, auditable, and token-efficient runtime instruction. This paper specifies how such a Skill should be structured: its input classes, output classes, internal phases, suitability gate, opcode dictionary, audit system, safety constraints, and final SKILL.md implementation architecture.

The core thesis is:

Requirement-to-Kernel conversion is a compilation problem, not a writing problem. (0.1)

The resulting Skill should therefore behave like a compiler that emits Kernel IR: a compact intermediate representation of the user's intent, suitable for stable LLM execution.



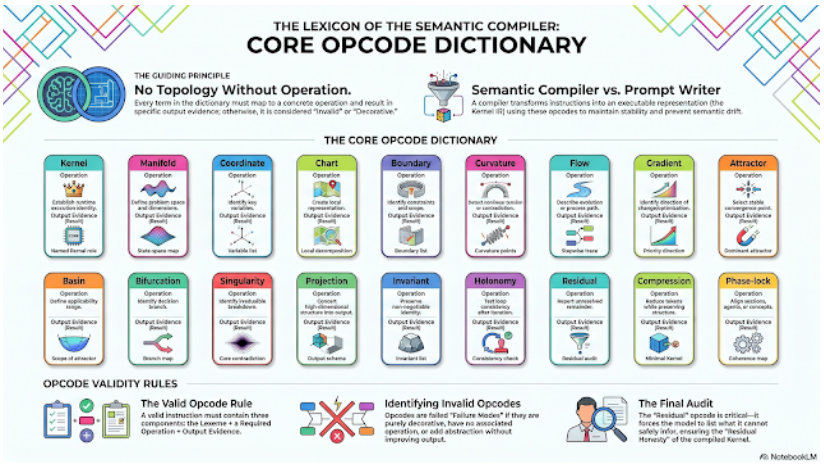


Table of Contents

1. Why This Skill Must Be a Compiler, Not a Prompt Writer
2. The Object Being Built: Differential-Topological Kernel Skill
3. Input Classes
4. Output Classes
5. The Full Compilation Pipeline
6. Phase 0: Suitability Gate — When Not to Use the Kernel
7. Phase 1: Intent Extraction

8. Phase 2: Boundary and Constraint Detection
9. Phase 3: Tension and Curvature Detection
10. Phase 4: Attractor Selection
11. Phase 5: Opcode Mapping
12. Phase 6: Kernel IR Composition
13. Phase 7: Compression and Token Budgeting
14. Phase 8: Stability, Safety, and Residual Audit
15. Phase 9: User-Facing Translation
16. Skill Output Templates
17. Worked Examples
18. Failure Modes
19. How the Final SKILL.md Should Be Structured
20. Conclusion
    Appendix A — Core Opcode Dictionary
    Appendix B — Minimal Kernel Patterns
    Appendix C — Audit Checklist

---

1. Why This Skill Must Be a Compiler, Not a Prompt Writer

A prompt writer takes an instruction and rewrites it into a better instruction.

A semantic compiler takes an instruction and transforms it into an executable representation.

This difference is decisive.

A normal prompt writer might receive:

> "Help me analyze this business requirement carefully."

and output:

> "You are a senior business analyst. Analyze the requirement step by step, identify goals, constraints, risks, and recommendations."

This is useful, but it remains ordinary prompt engineering. It uses role assignment, checklist expansion, and output formatting.

A Differential-Topological Kernel Skill should do something deeper. It should produce something closer to:

> "Run as Requirement Kernel. Map input into objective manifold; identify boundary conditions; detect curvature points where assumptions conflict; select dominant attractor; collapse into implementation trace; audit residual gaps."

This is not merely a nicer prompt. It is a compact runtime structure. It tells the model how to organize the problem-space, where to look for tension, how to choose a stable solution direction, and how to report what remains unresolved.

The transformation can be written as:

RawRequirement → IntentStructure → KernelIR → ExecutablePrompt. (1.1)

The Skill's true job is therefore:

SemanticCompiler(R) = K. (1.2)

Where:

R = raw requirement, article, long prompt, theory framework, or doctrine. (1.3)

K = compact Kernel prompt with opcode stack, boundary rules, and residual audit. (1.4)

This means the Skill should be evaluated not by whether its prompt sounds impressive, but by whether the generated Kernel is:

- shorter than the original requirement;
- more stable than ordinary prompting;
- faithful to the user's intent;
- resistant to semantic drift;
- auditable;
- reusable across similar tasks.

The Skill is not a style converter. It is a semantic compiler.

---

2. The Object Being Built: Differential-Topological Kernel Skill

The proposed Skill can be named:

Differential-Topological Kernel Generator

or more precisely:

Attractor-Lexeme Kernel Compiler

Its job is:

Input: loose semantic material. (2.1)

Output: compact Kernel prompt plus compilation trace. (2.2)

The Skill should convert:

- requirements into execution kernels;
- article frameworks into theory kernels;
- long prompts into minimal prompt kernels;
- domain doctrines into operating kernels;
- workflows into runtime kernels;
- analysis methods into reusable reasoning kernels.

A simplified definition:

Differential-Topological Kernel Skill := Semantic compiler that transforms natural-language intent into a compact attractor-opcode prompt. (2.3)

The Skill relies on three assumptions.

2.1 Assumption One: Kernel Is a Meta-Attractor

"Kernel" is not a neutral word. It evokes core execution, runtime authority, compact law, central operation, and non-casual procedural identity.

But this must be disambiguated. The Skill should not simply say:

> "You are the Kernel."

It should say:

> "Run as a Runtime Reasoning Kernel: a compact execution law for transforming input into stable structured output."

This gives the model a clearer execution posture.

KernelMetaAttractor := RuntimeIdentity + OperatingLaw + OutputContract. (2.4)

2.2 Assumption Two: Topological Lexemes Are Procedural Attractors

Terms such as **boundary**, **curvature**, **attractor**, **bifurcation**, and **residual** are not merely metaphors. They can function as compressed instructions.

For example:

boundary → identify scope, constraints, exclusions, and admissible region. (2.5)

curvature → detect nonlinear tension, distortion, or failure of simple framing. (2.6)

attractor → identify the dominant stable solution direction. (2.7)

residual → identify what remains unexplained or unresolved. (2.8)

Each lexeme must map to an operation.

ValidOpcode(L) := Lexeme + RequiredOperation + OutputEvidence. (2.9)

2.3 Assumption Three: The Skill Must Preserve Intent Before Compression

Compression without preservation is distortion.

Therefore, the Skill must first extract the user's intent before applying topology.

IntentPreservation > TopologicalElegance. (2.10)

This is a core rule.

A bad Skill decorates every task with geometric language.

A good Skill first asks internally:

> What is the user truly trying to achieve?

Only after that should it compile.

---

3. Input Classes

The Skill should handle at least five input classes.

3.1 Input Class A: Practical Requirement

Example:

> “Build a system that extracts timelines from legal documents.”

This type requires objective extraction, source identification, output definition, constraints, and risk analysis.

Compilation goal:

PracticalRequirement → ExecutionKernel. (3.1)

3.2 Input Class B: Theory Framework

Example:

> “This article argues that AI systems need observer-like runtime structures to stabilize self-correction.”

This type requires thesis extraction, concept hierarchy, assumptions, tension fields, and framework compression.

Compilation goal:

TheoryFramework → ConceptualKernel. (3.2)

3.3 Input Class C: Long Prompt

Example:

> A 2,000-token prompt describing a role, task, workflow, formatting rules, examples, and quality checks.

This type requires redundancy removal, rule hierarchy, opcode mapping, and token reduction.

Compilation goal:

LongPrompt → MinimalKernel. (3.3)

3.4 Input Class D: Domain Doctrine

Example:

> A company policy, management doctrine, legal test, clinical guideline, or accounting standard.

This type requires invariant extraction, boundary detection, decision branch mapping, and output trace design.

Compilation goal:

DomainDoctrine → GovernanceKernel. (3.4)

3.5 Input Class E: Hybrid Framework

Example:

> A theory article plus practical implementation requirement.

This type requires two-layer compilation:

HybridInput → TheoryKernel + ExecutionKernel + BridgeKernel. (3.5)

This is important for advanced use. Many real tasks require translating a theory into an operating method.

---

4. Output Classes

The Skill should not output only one prompt. It should emit a package.

SkillOutput := FullKernel + MinimalKernel + OpcodeMap + CompressionTrace + ResidualAudit. (4.1)

4.1 Full Kernel

The Full Kernel is a readable, safer, expanded version. It should include:

- runtime identity;
- task objective;
- opcode stack;
- boundary rules;
- output contract;
- residual audit instruction.

Example:

> “Run as Requirement Kernel. Interpret the input as a problem manifold. Extract objective coordinates, boundary conditions, curvature points, dominant attractor, feasible action trace, and residual gaps. Do not invent constraints. Preserve user intent. Output: summary, kernel map, action trace, risks, residuals.”

4.2 Minimal Kernel

The Minimal Kernel is token-efficient. It may be used inside system prompts, reusable prompt snippets, or workflow agents.

Example:

> “Run as ReqKernel: map manifold; scan boundary; detect curvature; select attractor; collapse action trace; audit residuals.”

The Minimal Kernel is not meant to explain itself. It is meant to run.

4.3 Opcode Map

The Opcode Map explains why each lexeme appears.

Example:

Opcode	Reason
manifold	requirement has multiple interacting dimensions
boundary	legal and data-source constraints matter
curvature	date conflicts and incomplete evidence may distort timeline
attractor	output must converge to one ordered trace
residual	unsupported or missing events must be reported

4.4 Compression Trace

The Compression Trace shows what was preserved and what was compressed.

CompressionTrace := OriginalIntent + PreservedCore + CompressedElements + DroppedNoise + ResidualRisk. (4.2)

This is essential for trust.

4.5 Residual Audit

The Skill must report what the Kernel cannot safely encode.

Examples:

- missing domain-specific rules;
- unclear output audience;
- unknown safety constraints;
- ambiguous authority hierarchy;
- possible over-abstraction.

ResidualAudit := UnresolvedInputs + AmbiguousAssumptions + KernelLimitations. (4.3)

---

5. The Full Compilation Pipeline

The Skill should follow a fixed internal pipeline.

Pipeline := Gate → Intent → Boundary → Curvature → Attractor → Opcode → KernelIR → Compression → Audit → Translation. (5.1)

Expanded:

1. Suitability Gate
2. Intent Extraction
3. Boundary Detection
4. Tension / Curvature Detection
5. Attractor Selection
6. Opcode Mapping
7. Kernel IR Composition
8. Token Compression
9. Stability and Residual Audit
10. User-Facing Translation

This sequence matters.

A bad compiler jumps directly from input to fancy Kernel.

A good compiler first checks whether Kernel conversion is justified.

---

6. Phase 0: Suitability Gate — When Not to Use the Kernel

The first phase must decide whether Differential-Topological Kernel conversion is appropriate.

Not every task needs it.

6.1 When the Kernel Is Useful

Kernel conversion is useful when the task has:

- multiple constraints;
- ambiguity;
- cross-domain mapping;
- need for stable repeated reasoning;
- theoretical framework compression;
- long prompt reduction;
- risk of drift;
- hidden tensions;
- need for reusable execution structure.

KernelNeed := Complexity + Ambiguity + ConstraintLoad + CrossDomainMapping + StabilityNeed. (6.1)

Use Kernel if:

KernelNeed > SimplicityThreshold. (6.2)

6.2 When the Kernel Is Not Useful

The Skill should avoid Kernel conversion when the task is:

- simple rewriting;
- basic translation;
- direct factual Q&A;
- purely stylistic editing;
- low-risk formatting;
- already sufficiently structured.

Example:

> “Rewrite this email politely.”

A good Skill should say:

> “A Differential-Topological Kernel is unnecessary. Use a plain rewrite prompt.”

This anti-overuse gate is essential.

6.3 Suitability Output

The Skill should internally classify:

Suitability := {UseKernel, UsePlainPrompt, UseHybrid}. (6.3)

Where:

- **UseKernel** = generate full Kernel package;
- **UsePlainPrompt** = output ordinary structured prompt;
- **UseHybrid** = use small Kernel internally but produce simple user-facing prompt.

---

7. Phase 1: Intent Extraction

Before topology, extract intent.

The Skill must identify:

- user objective;
- target output;
- audience;
- domain;
- required depth;
- constraints;

- success criteria;
- implied risks.

IntentCore := Objective + Domain + OutputNeed + Audience + SuccessCriteria. (7.1)

For example, input:

> “Convert this theory article into a reusable prompt framework for AI engineers.”

Intent extraction:

- Objective: convert theory into prompt framework.
- Domain: AI engineering.
- Output: reusable prompt / Skill structure.
- Audience: AI engineers.
- Success criteria: compact, executable, stable, understandable.
- Risk: over-abstraction, loss of theory nuance.

Intent extraction must happen before opcode selection.

Otherwise, topology words may distort the task.

---

8. Phase 2: Boundary and Constraint Detection

Boundary detection identifies what must not be crossed.

Boundary := Scope + Constraints + Exclusions + AuthorityHierarchy + SafetyLimits. (8.1)

The Skill should detect at least five boundary types.

8.1 Scope Boundary

What is included and excluded?

Example:

- Include: requirement parsing and Kernel generation.
- Exclude: empirical benchmark execution.

8.2 Domain Boundary

Which domain language applies?

Example:

- Legal document review uses evidence, claims, dates, sources.
- Theory article conversion uses thesis, assumptions, concept maps.

8.3 Authority Boundary

What instructions dominate?

AuthorityHierarchy := SystemRules > DeveloperRules > SafetyRules > UserIntent > KernelPrompt > Formatting. (8.2)

The Kernel must never override higher instructions.

8.4 Output Boundary

What output is expected?

Examples:

- prompt only;
- prompt plus explanation;
- full Skill design;
- SKILL.md-ready content;
- testing checklist.

8.5 Risk Boundary

What should the Kernel avoid?

Examples:

- hallucinated constraints;

- unsafe authority escalation;
- fake mathematical rigor;
- decorative jargon;
- over-compression.

The Skill should emit boundary findings in the trace.

---

9. Phase 3: Tension and Curvature Detection

Curvature detection is one of the most important phases.

In this context, **curvature** means:

> where the requirement is not flat; where simple interpretation fails; where hidden tension, contradiction, nonlinearity, or phase mismatch appears.

Curvature := NonlinearTension + Contradiction + Ambiguity + HiddenDependency. (9.1)

The Skill should look for several curvature types.

9.1 Objective Curvature

The user asks for two goals that may conflict.

Example:

> “Make it very short but also complete.”

Curvature:

Completeness ↔ Token minimality. (9.2)

9.2 Domain Curvature

The framework crosses domains with different assumptions.

Example:

> “Use differential topology terms for LLM prompt engineering.”

Curvature:

Mathematical rigor ↔ symbolic prompt utility. (9.3)

9.3 Execution Curvature

The output must both guide reasoning and remain readable.

Curvature:

Internal Kernel density ↔ external user clarity. (9.4)

9.4 Safety Curvature

The word “Kernel” may imply authority, but must remain subordinate.

Curvature:

Runtime identity ↔ instruction hierarchy safety. (9.5)

9.5 Evidence Curvature

The method is plausible but not empirically proven.

Curvature:

Strong theory framing ↔ limited benchmark evidence. (9.6)

The Skill should not hide curvature. It should report it and encode it into the Kernel.

A mature Kernel does not erase tension; it stabilizes it.

---

10. Phase 4: Attractor Selection

After detecting boundaries and curvature, the Skill must choose the dominant attractor.

An attractor is the stable direction the Kernel should collapse toward.

Attractor := DominantStablePurpose + OutputConvergencePoint. (10.1)

Examples:

Input Type	Dominant Attractor
User requirement	executable solution
Theory article	conceptual framework
Long prompt	minimal runtime instruction
Policy doctrine	decision procedure
Research idea	testable thesis
Skill design	repeatable workflow

The Skill should identify:

- primary attractor;
- secondary attractors;
- rejected attractors;
- risk of wrong attractor selection.

Example:

Input:

> “Write a Skill that converts theories into topology kernels.”

Possible attractors:

1. Prompt generator
2. Semantic compiler
3. Academic article writer
4. Skill authoring assistant

The correct attractor is:

SemanticCompiler. (10.2)

This is why the Skill must choose carefully.

Wrong attractor selection produces wrong Kernel architecture.

---

11. Phase 5: Opcode Mapping

Opcode mapping converts semantic findings into kernel lexemes.

OpcodeMap := Findings → ProceduralLexemes. (11.1)

For example:

Finding	Opcode
Multi-dimensional problem	manifold
Constraints matter	boundary
Hidden contradiction	curvature
Need convergence	attractor
Branch decisions	bifurcation
Need transformation	projection
Need consistency after loop	holonomy
Need leftover audit	residual
Need preserved identity	invariant
Need repeated process	flow

Every opcode must have an operation.

11.1 Valid Opcode Rule

ValidOpcode := Lexeme + Operation + Evidence. (11.2)

Example:

boundary → identify scope and constraints → output boundary list. (11.3)

curvature → identify nonlinear tension → output tension points. (11.4)

attractor → choose stable objective direction → output selected attractor. (11.5)

residual → report unresolved gaps → output residual audit. (11.6)

11.2 Invalid Opcode Rule

An opcode is invalid if:

- it has no operation;
- it is decorative;

- it confuses the model;
- it adds unnecessary abstraction;
- it does not improve the output.

InvalidOpcode := Lexeme – Operation. (11.7)

The future Skill should remove invalid opcodes.

---

12. Phase 6: Kernel IR Composition

Kernel IR is the intermediate representation produced before final prompt compression.

KernelIR := RuntimeIdentity + Objective + OpcodeStack + BoundaryRules + OutputContract + ResidualAudit. (12.1)

12.1 Runtime Identity

Example:

> "Run as Requirement-to-Kernel Compiler."

or:

> "Run as Theory Kernel Compiler."

This is more precise than merely saying "You are an assistant."

12.2 Objective

The objective should be explicit.

Example:

> "Transform the input requirement into a compact reusable Kernel prompt."

12.3 Opcode Stack

The opcode stack should be ordered.

Example:

Map manifold → detect boundary → scan curvature → select attractor → project Kernel → audit residual. (12.2)

Order matters. A shuffled list is weaker than a procedural chain.

12.4 Boundary Rules

Example:

> "Do not invent constraints. Do not use topology terms decoratively. Preserve original user intent."

12.5 Output Contract

Example:

> "Output Full Kernel, Minimal Kernel, Opcode Map, Compression Trace, Residual Audit."

12.6 Residual Audit

Example:

> "List what the Kernel cannot safely infer from the input."

A complete Kernel IR might look like:

```
Run as Requirement-to-Kernel Compiler.
Objective: convert raw requirement into stable Differential-Topological Kernel.
Process: extract intent → map manifold → detect boundary → scan curvature → select attractor → compose opcode stack → compress Kernel → audit residual.
Rules: preserve user intent; no decorative topology; do not override higher instructions; report ambiguity.
Output: Full Kernel, Minimal Kernel, Opcode Map, Compression Trace, Residual Audit.
```

This is not yet the most minimal form. It is the safe IR.

---

13. Phase 7: Compression and Token Budgeting

After Kernel IR is composed, the Skill should generate shorter versions.

Compression should preserve structure.

Compression := RemoveRedundancy + PreserveOpcodeOrder + PreserveBoundary + PreserveAudit. (13.1)

13.1 Full Kernel

Readable and safe:

```
Run as Requirement-to-Kernel Compiler. Extract the user's intent,
map the requirement into a problem manifold, identify boundary conditions,
detect curvature points, select the dominant attractor, compose a topology opcode stack,
collapse it into a reusable Kernel prompt, and audit residual gaps.
Preserve user intent. Do not use topology terms decoratively.
```

13.2 Compact Kernel

Shorter but still clear:

```
Run as Req→Kernel Compiler: intent → manifold → boundary → curvature → attractor → opcode stack
→ Kernel prompt → residual audit. Preserve intent; no decorative topology.
```

13.3 Minimal Kernel

Very short:

```
ReqKernel: intent→manifold→boundary→curvature→attractor→kernel→residual.
Preserve intent; no decoration.
```

13.4 Token Compression Risk

Compression can lose safety.

The Skill should track:

CompressionRisk := LostBoundary + LostObjective + LostAudit + AmbiguousOpcode. (13.2)

If risk becomes too high, the Skill should prefer compact rather than minimal.

---

14. Phase 8: Stability, Safety, and Residual Audit

The Skill must audit the generated Kernel before final output.

Audit := StabilityAudit + SafetyAudit + ResidualAudit + OverTopologyAudit. (14.1)

14.1 Stability Audit

Questions:

- Does the Kernel have a clear runtime identity?
- Is the opcode order logical?
- Is there a stable output contract?
- Is the residual audit preserved?
- Would repeated runs likely produce similar structure?

Stability(K) := Repeatability + StructureClarity + DriftResistance. (14.2)

14.2 Safety Audit

Questions:

- Does the Kernel claim too much authority?
- Does it override instruction hierarchy?
- Does it encourage hidden reasoning disclosure?
- Does it invite unsafe compliance?
- Does it create false rigor?

Safety(K) := BoundaryRespect + NonEscalation + HonestLimitations. (14.3)

14.3 Residual Audit

Questions:

- What information is missing?
- What assumptions are uncertain?
- What domain-specific knowledge is not encoded?
- What is left out by compression?

Residual(K) := MissingInfo + Ambiguity + UnencodedContext + FutureWork. (14.4)

14.4 Over-Topology Audit

Questions:

- Are topology terms actually needed?
- Are any terms decorative?
- Can ordinary words do the job better?
- Is the output too abstract for the user?

OverTopologyRisk := DecorativeLexemes + UnnecessaryAbstraction + UserConfusion. (14.5)

The Skill should explicitly remove decorative terms.

---

15. Phase 9: User-Facing Translation

The Skill should distinguish internal Kernel language from user-facing language.

InternalKernelLanguage := topology-rich, compact, high-attractor. (15.1)

ExternalUserLanguage := clear, domain-readable, explanation-oriented. (15.2)

Example:

Internal:

> “Detect curvature.”

External:

> “Find hidden contradictions or nonlinear tensions in the requirement.”

Internal:

> “Select attractor.”

External:

> “Identify the main stable direction the final output should converge toward.”

Internal:

> “Audit residual.”

External:

> “List what remains unresolved or unsupported.”

The Skill may output both.

This is important because a Kernel is meant for LLM execution, but the user needs to understand what was compiled.

---

16. Skill Output Templates

The Skill should use predictable templates.

16.1 Full Output Template

```
# Kernel Conversion Result

## 1. Suitability
UseKernel / UsePlainPrompt / UseHybrid
Reason: ...

## 2. Extracted Intent
Objective:
Domain:
Output Need:
Audience:
Success Criteria:

## 3. Boundary Conditions
Scope:
Constraints:
Exclusions:
Safety / Authority Notes:

## 4. Curvature Points
Tension 1:
Tension 2:
Tension 3:

## 5. Dominant Attractor
Selected Attractor:
Rejected Attractors:
Reason:

## 6. Opcode Map
Opcode:
Operation:
Reason:

## 7. Full Kernel
```

```
...

## 8. Minimal Kernel
...

## 9. Compression Trace
Preserved:
Compressed:
Dropped:
Uncertain:

## 10. Residual Audit
...
```

16.2 Short Output Template

```
Suitability: ...
Intent: ...
Opcode Stack: ...
Full Kernel: ...
Minimal Kernel: ...
Residuals: ...
```

16.3 Minimal Output Template

```
Kernel:
[Minimal Kernel]

Trace:
Intent → Boundary → Curvature → Attractor → Residual
```

---

17. Worked Examples

17.1 Example A — Practical Requirement

Input:

> “I need a prompt that helps an AI review contracts and find risky clauses.”

Suitability

UseKernel.

Reason: legal review involves constraints, risk boundaries, evidence, and residual uncertainty.

Extracted Intent

Objective: review contracts for risky clauses.
Domain: legal / contract analysis.
Output: clause risk report.
Success: identify risks without inventing legal conclusions.
Risk: hallucinated legal advice.

Boundary

- Do not provide final legal judgment.
- Cite clause text when possible.
- Separate risk flag from legal conclusion.
- Report uncertainty.

Curvature

- User wants strong risk detection but legal certainty may be unavailable.
- Contract language may be ambiguous.
- Different jurisdictions may change meaning.

Attractor

Dominant attractor: evidence-based risk triage.

Opcode Stack

contract manifold → clause boundary → ambiguity curvature → risk attractor → evidence projection → residual audit. (17.1)

Full Kernel

```
Run as Contract Risk Kernel. Map the contract into a clause manifold;
identify scope and jurisdiction boundaries;
detect curvature where wording creates ambiguity, obligation imbalance, missing definitions,
hidden liability, or enforcement uncertainty;
select risk attractors by severity and likelihood;
project findings into an evidence-based clause report; audit residual legal uncertainties.
Do not invent legal conclusions. Separate risk flags from legal advice.
```

Minimal Kernel

```
ContractRiskKernel:
clause manifold→boundary→ambiguity curvature→risk attractor→evidence report→legal residuals.
No invented conclusions.
```

17.2 Example B — Theory Article

Input:

> “Convert my article on Observer Thinning into a reusable prompt framework.”

Suitability

UseKernel.

Reason: theory-to-framework conversion requires abstraction, compression, and preservation of conceptual invariants.

Intent

Objective: convert theory into reusable prompt framework.
Domain: AI cognition / observer theory.
Output: prompt framework.
Success: preserve core theory while making it executable.

Boundary

- Preserve central thesis.
- Avoid metaphysical overclaim.
- Translate theory into operations.

Curvature

- Theory language may be rich but not directly executable.
- Prompt must be compact but conceptually faithful.
- Observer terms may become vague if not operationalized.

Attractor

Dominant attractor: executable observer-diagnostics framework.

Opcode Stack

theory manifold → invariant extraction → observer boundary → thinning curvature → diagnostic attractor → prompt projection → residual audit. (17.2)

Full Kernel

```
Run as Theory-to-Prompt Kernel. Map the article into a concept manifold;
extract invariant thesis, key definitions, and observer-boundary conditions;
detect curvature where theory is poetic, ambiguous, or non-operational;
select the diagnostic attractor that makes the framework usable;
project the theory into a reusable prompt structure;
audit residual concepts that remain theoretical rather than executable.
```

Minimal Kernel

```
TheoryPromptKernel:
concept manifold→invariants→observer boundary→curvature→diagnostic attractor→prompt projection→residuals.
```

17.3 Example C — Long Prompt Compression

Input:

> A long prompt telling the model to analyze business requirements, identify constraints, ask clarifying questions, produce implementation steps, and report risks.

Suitability

UseHybrid.

Reason: long prompt can be compressed into Kernel, but final output should remain practical.

Attractor

Dominant attractor: requirement analysis workflow.

Opcode Stack

intent coordinates → boundary scan → curvature detection → implementation attractor → action projection → residual questions. (17.3)

Minimal Kernel

```
BizReqKernel: intent coords→boundary→curvature→implementation attractor→action plan→residual questions.
```

---

18. Failure Modes

A serious Skill must know its own failure modes.

18.1 Decorative Topology

Symptom:

The Kernel contains fancy terms but no executable operations.

Example:

```
Use manifold holonomy curvature to deeply understand the system.
```

Fix:

Every lexeme must map to an operation.

18.2 Over-Compression

Symptom:

The minimal Kernel is too short and loses boundary rules.

Example:

```
Kernel: manifold→attractor→output.
```

Fix:

Preserve boundary and residual.

18.3 Wrong Attractor

Symptom:

The Kernel optimizes for the wrong output.

Example:

User wants a Skill, but Kernel produces an article.

Fix:

Attractor selection must distinguish output class.

18.4 Kernel Authority Misfire

Symptom:

The Kernel sounds like it overrides safety or system rules.

Fix:

Always include hierarchy subordination.

18.5 User Confusion

Symptom:

User cannot understand the generated Kernel.

Fix:

Provide user-facing translation.

18.6 Topology Where Plain Prompt Is Better

Symptom:

Simple task becomes unnecessarily complex.

Fix:

Suitability gate.

---

19. How the Final SKILL.md Should Be Structured

The final Skill should be written as an operational guide, not as an essay.

Recommended SKILL.md structure:

```
# Differential-Topological Kernel Generator
```

```
## Purpose
Convert complex requirements, theory frameworks, long prompts,
or doctrines into compact Kernel prompts using validated topology-inspired opcodes.

## When to Use
Use when the input is complex, ambiguous, multi-constraint, cross-domain, theory-heavy,
or requires reusable stable reasoning.

## When Not to Use
Do not use for simple rewriting, translation, direct factual answers, low-risk formatting,
or tasks where topology terms would be decorative.

## Core Principle
Act as a semantic compiler, not a prompt decorator.

## Pipeline
1. Suitability Gate
2. Intent Extraction
3. Boundary Detection
4. Curvature Detection
5. Attractor Selection
6. Opcode Mapping
7. Kernel IR Composition
8. Compression
9. Audit
10. User-Facing Translation

## Opcode Rules
Every topology term must map to a concrete operation and output evidence.

## Output Formats
Full Kernel
Minimal Kernel
Opcode Map
Compression Trace
Residual Audit

## Safety Rules
Do not override instruction hierarchy.
Do not invent constraints.
Do not use decorative topology.
Preserve user intent.
Report uncertainty.

## Examples
...
```

The final Skill must be short enough to be usable, but detailed enough to enforce the method.

The most important sentence in the Skill should be:

```
You are a semantic compiler: preserve user intent, map it into valid topology opcodes,
compose a compact Kernel, and audit residual loss.
```

---

20. Conclusion

A Skill for Differential-Topological Kernel conversion should be designed as a **semantic compiler**.

Its job is not merely to rewrite prompts. It must transform loose semantic material into a compact runtime structure.

The correct transformation is:

RequirementSource → SemanticCompiler → KernelIR → ExecutableKernel → AuditedOutput. (20.1)

The Skill must perform:

- suitability gating;
- intent preservation;
- boundary detection;
- curvature detection;
- attractor selection;
- opcode mapping;
- Kernel IR composition;
- token compression;
- stability audit;
- user-facing translation.

Its central discipline is:

No topology without operation. No compression without preservation. No Kernel without residual audit. (20.2)

The eventual SKILL.md should therefore not be written as a collection of clever prompt tricks. It should be written as a compact compiler specification.

This is the bridge from theory to tool.

The first article established the conceptual claim:

Kernel + topology lexemes can act as a two-level attractor system. (20.3)

This second article establishes the engineering claim:

A Skill can compile requirements and theories into that attractor system through a disciplined pipeline. (20.4)

The third step will be to write the actual Skill.

Appendix A — Core Opcode Dictionary

Opcode	Operation	Output Evidence
Kernel	Establish runtime execution identity	Named Kernel role
Manifold	Define problem space and dimensions	State-space map
Coordinate	Identify key variables	Variable list
Chart	Create local representation	Local decomposition
Boundary	Identify constraints and scope	Boundary list
Curvature	Detect nonlinear tension or contradiction	Curvature points
Flow	Describe evolution or process path	Stepwise trace
Gradient	Identify direction of change or optimization	Priority direction
Attractor	Select stable convergence point	Dominant attractor
Basin	Define applicability range	Scope of attractor
Bifurcation	Identify decision branch	Branch map
Singularity	Identify irreducible breakdown	Core contradiction
Projection	Convert high-dimensional structure into output	Output schema
Invariant	Preserve non-negotiable identity	Invariant list
Holonomy	Test loop consistency after iteration	Consistency check
Residual	Report unresolved remainder	Residual audit
Compression	Reduce tokens while preserving structure	Minimal Kernel
Phase-lock	Align sections, agents, or concepts	Coherence map

Appendix B — Minimal Kernel Patterns

B.1 Requirement Kernel

```
ReqKernel: intent→manifold→boundary→curvature→attractor→action trace→residual.
Preserve intent; no decoration.
```

B.2 Theory Kernel

```
TheoryKernel: thesis→concept manifold→invariants→curvature→attractor→projection→residual.
```

B.3 Prompt Compression Kernel

```
PromptKernel: intent→rules→boundary→opcode stack→minimal kernel→loss audit.
```

B.4 Workflow Kernel

```
WorkflowKernel: objective→state manifold→boundary→flow→bifurcation→output trace→residual.
```

B.5 Risk Analysis Kernel

```
RiskKernel: scope→boundary→curvature→risk attractors→severity projection→residual uncertainty.
```

Appendix C — Audit Checklist

C.1 Suitability Audit

```
Is the task complex enough for Kernel conversion?
Is there ambiguity?
Are there multiple constraints?
Is repeatable reasoning needed?
Would ordinary prompting be enough?
```

C.2 Intent Audit

```
Is the user objective preserved?
Is the output type clear?
Is the audience clear?
Are success criteria identified?
```

C.3 Opcode Audit

```
Does every topology term map to an operation?
Is any term decorative?
Is opcode order logical?
Is the stack too long?
```

C.4 Boundary Audit

```
Are scope limits clear?
Are safety and authority boundaries preserved?
Are exclusions stated?
Does the Kernel avoid invented constraints?
```

C.5 Compression Audit

```
Was important intent lost?
Was residual audit preserved?
Did token reduction create ambiguity?
Is the minimal Kernel still executable?
```

C.6 Stability Audit

```
Would repeated runs produce similar structure?
Does the Kernel reduce drift?
Does it force premature collapse?
Does it include residual reporting?
```

C.7 Final Quality Formula

KernelQuality := IntentPreservation + Executability + Stability + Minimality + ResidualHonesty − DecorativeTopology − DriftRisk − AuthorityMisfire. (C.1)

© 2026 Danny Yeung. All rights reserved. 版权所有 不得转载

Disclaimer

This book is the product of a collaboration between the author and OpenAI's GPT-5.4, X's Grok, Google Gemini 3, NotebookLM, Claude's Sonnet 4.6, Haiku 4.5 language model. While every effort has been made to ensure accuracy, clarity, and insight, the content is generated with the assistance of artificial intelligence and may contain factual, interpretive, or mathematical errors. Readers are encouraged to approach the ideas with critical thinking and to consult primary scientific literature where appropriate.

This work is speculative, interdisciplinary, and exploratory in nature. It bridges metaphysics, physics, and organizational theory to propose a novel conceptual framework—not a definitive scientific theory. As such, it invites dialogue, challenge, and refinement.

I am merely a midwife of knowledge.